

Tarefa 10 – Mecanismos de Sincronização com OpenMP

Israel Soares de Castro Filho
Matrícula: 20250070780

1 Introdução

Este relatório documenta a Tarefa 10, cujo objetivo é estudar, de forma prática, os principais mecanismos de sincronização disponíveis no OpenMP para proteger uma variável compartilhada em regiões paralelas. Como estudo de caso, mantém-se o problema já utilizado na Tarefa 8: a estimativa estocástica de π pelo método de Monte Carlo, no qual cada thread sorteia pontos (x, y) no quadrado $[0, 1] \times [0, 1]$ e conta quantos caem dentro do quarto de círculo de raio unitário. A razão entre acertos e total de pontos, multiplicada por 4, converge para π .

O foco desta tarefa não é a precisão da estimativa em si, mas sim **como** a contagem de acertos é somada entre as threads e o impacto disso no tempo de execução. Foram implementadas e comparadas cinco versões do estimador, variando a forma de acumulação do contador de acertos e o mecanismo de exclusão mútua utilizado.

2 Metodologia

A partir do código de referência da Tarefa 8, foram implementadas 5 versões da função de estimativa de π , todas paralelizadas com `#pragma omp parallel` e `#pragma omp for`, variando apenas a estratégia de acumulação do contador de acertos:

- **V1 – privado + rand() + critical:** cada thread mantém um contador local (`acertos_local`) e soma seu valor à variável global apenas **uma vez**, ao final da região paralela, dentro de uma seção `#pragma omp critical`. Utiliza o gerador `rand()` da biblioteca padrão (não thread-safe, herdado da Tarefa 8 como baseline de referência).
- **V2 – privado + xorshift32 + critical:** idêntico a V1 na estratégia de sincronização (contador privado, um único `critical` por thread ao final), mas usando o gerador Xorshift32 com estado privado por thread em vez de `rand()`.
- **V3 – compartilhado + xorshift32 + critical:** o contador de acertos é uma única variável **compartilhada**, incrementada dentro de uma seção `#pragma omp critical` a **cada acerto** (isto é, potencialmente milhões de entradas na seção crítica, e não apenas uma por thread).
- **V4 – compartilhado + xorshift32 + atomic:** mesma estrutura de V3 (contador compartilhado, atualizado a cada acerto), trocando a diretiva `critical` pela diretiva `#pragma omp atomic`.

- **V5 – xorshift32 + reduction:** nenhuma diretiva de exclusão mútua é escrita manualmente; o acúmulo é delegado à cláusula `reduction(+:total_acertos)` do próprio `#pragma omp for`, que cria cópias privadas do contador para cada thread e as combina automaticamente ao final do laço.

Observação sobre o gerador aleatório. O enunciado original da tarefa pedia o uso de `rand_r()` (gerador thread-safe do padrão POSIX) nas versões com contador compartilhado (V3 e V4). Como o ambiente de desenvolvimento e testes utilizado é o Windows, onde `rand_r()` não está disponível de forma padronizada, essa chamada foi substituída pela função `random_double()`, baseada no gerador Xorshift32 com estado (`seed`) privado por thread – a mesma estratégia já adotada na Tarefa 8 para contornar a mesma limitação. Do ponto de vista do experimento, essa troca não compromete a análise: o objetivo da tarefa é comparar o **custo dos mecanismos de sincronização** (critical, atomic, reduction), e não o gerador de números aleatórios em si; usar o mesmo gerador (Xorshift32) em V2, V3, V4 e V5 tem inclusive a vantagem de isolar melhor essa variável, deixando a diferença de desempenho atribuível quase exclusivamente à estratégia de sincronização escolhida.

Todas as versões foram executadas para o mesmo conjunto de tamanhos de entrada ($n_{\text{pontos}} \in \{10^4, 10^5, 10^6, 10^7, 10^8, 10^9\}$), com 20 threads, medindo o tempo de execução com `omp_get_wtime()` e o erro absoluto/relativo em relação a π real. O código-fonte completo encontra-se em `../src/tarefa10.c` e é reproduzido no Apêndice A.

3 Resultados

A Tabela 1 reproduz a saída obtida na execução do programa (arquivo `output.txt`), com $\pi_{\text{real}} = 3.141592653589793$ e 20 threads.

N_PONTOS	Versão	π medido	Tempo (s)
10.000	V1	3,11920000	0,0000
10.000	V2	3,14320000	0,0000
10.000	V3	3,14320000	0,0010
10.000	V4	3,14320000	0,0000
10.000	V5	3,14320000	0,0000
100.000	V1	3,10040000	0,0010
100.000	V2	3,14408000	0,0000
100.000	V3	3,14408000	0,0040
100.000	V4	3,14408000	0,0030
100.000	V5	3,14408000	0,0000
1.000.000	V1	3,13557200	0,0030
1.000.000	V2	3,14235200	0,0000
1.000.000	V3	3,14235200	0,0390
1.000.000	V4	3,14235200	0,0120
1.000.000	V5	3,14235200	0,0000
10.000.000	V1	3,14204040	0,0090
10.000.000	V2	3,14160360	0,0020
10.000.000	V3	3,14160360	0,3640
10.000.000	V4	3,14160360	0,1210
10.000.000	V5	3,14160360	0,0020
100.000.000	V1	3,14124624	0,0910
100.000.000	V2	3,14187084	0,0210
100.000.000	V3	3,14187084	3,7180
100.000.000	V4	3,14155864	1,1660
100.000.000	V5	3,14169476	0,0220
1.000.000.000	V1	3,14137154	0,9300
1.000.000.000	V2	3,14161538	0,2110
1.000.000.000	V3	3,14161538	38,6540
1.000.000.000	V4	3,14157180	11,8630
1.000.000.000	V5	3,14160934	0,2110

Tabela 1: Comparação de π medido e tempo de execução entre as 5 versões, para 20 threads (extraído de `output.txt`).

Para facilitar a leitura, a Tabela 2 isola o caso de maior carga ($n_{\text{pontos}} = 10^9$), tomando como referência a versão mais lenta (V3, compartilhado + critical).

Versão	Tempo (s)	Speedup em relação a V3
V3 – compartilhado + critical	38,6540	1,0× (referência)
V4 – compartilhado + atomic	11,8630	$\approx 3,3\times$
V1 – privado + critical (rand)	0,9300	$\approx 41,6\times$
V2 – privado + critical (xorshift)	0,2110	$\approx 183,2\times$
V5 – reduction	0,2110	$\approx 183,2\times$

Tabela 2: Comparação de tempo para $n_pontos = 1.000.000.000$.

4 Análise

4.1 Comparação entre as cinco versões

Os resultados da Tabela 1 deixam claro que o erro de estimativa (absoluto e relativo) é, como esperado, praticamente idêntico entre V2, V3, V4 e V5 para um mesmo n_pontos – afinal, todas usam o mesmo gerador (Xorshift32) e a mesma quantidade de pontos sorteados; pequenas diferenças de π medido entre elas (visíveis a partir de $n_pontos = 10^8$) vêm apenas da ordem de acumulação das somas parciais em ponto flutuante/inteiro combinada com a forma como as sementes de cada thread são geradas, e não da qualidade do mecanismo de sincronização. V1 se destaca com erro maior porque usa `rand()`, que tem baixa qualidade estatística e, principalmente, não é thread-safe quando chamado concorrentemente por várias threads sem proteção – o que já era uma limitação conhecida herdada da Tarefa 8.

O ponto central da tarefa, porém, é o **tempo de execução**, e aí as diferenças são drásticas:

- **V3 (compartilhado + critical) é, de longe, a pior versão.** Para $n_pontos = 10^9$ ela leva 38,65s, contra 0,21s de V2/V5 – quase 200 vezes mais lenta. Isso acontece porque a seção crítica é disputada por até 20 threads **a cada acerto** (cerca de 78,5% dos pontos sorteados caem dentro do círculo), e não apenas uma vez por thread. Cada entrada em `critical` envolve a aquisição de um mutex genérico, o que serializa fortemente as threads e faz o custo de sincronização dominar completamente o tempo total.
- **V4 (compartilhado + atomic) melhora bastante em relação a V3** (11,86s vs. 38,65s, um ganho de $\approx 3,3\times$), pois a diretiva `atomic` usa uma instrução atômica de hardware em vez de um mutex de propósito geral, com overhead bem menor para uma operação simples como `total_acertos++`. Ainda assim, é muito mais lenta que as versões com contador privado, pois a variável continua sendo compartilhada e disputada a cada incremento, gerando contenção na mesma linha de cache (*true sharing*) entre todas as threads.
- **V1 e V2 (contador privado + critical apenas ao final)** têm desempenho ordens de magnitude melhor que V3/V4, porque a seção crítica é executada apenas $O(n^0)$ de threads vezes (uma soma por thread ao final da região paralela), e não $O(n_pontos)$ vezes. Isso confirma que o problema de V3/V4 não é o uso de `critical/atomic` em si, mas a **frequência** com que a sincronização é acionada.

- **V5 (reduction) tem desempenho equivalente ao melhor caso manual (V2)**, sem que o programador precise escrever nenhuma sincronização explícita. O compilador implementa a redução criando cópias privadas do contador por thread (evitando qualquer disputa durante o laço) e combinando-as ao final, de forma tipicamente mais eficiente ou pelo menos equivalente à melhor estratégia manual.

Em suma, a ordem de desempenho observada foi:

V3 (critical/iteração) $\ll$ V4 (atomic/iteração) $\ll$ V1 $\approx$ V2 $\approx$ V5 (reduction).

4.2 Reflexão: aplicabilidade em desempenho e produtividade

Os mecanismos de sincronização do OpenMP não são substitutos equivalentes entre si – cada um representa um compromisso diferente entre **desempenho**, **generalidade** e **produtividade/simplicidade do código**:

- **reduction** é o mecanismo de **maior produtividade e, para o caso testado, também de melhor desempenho**: o programador não escreve nenhum código de sincronização, elimina uma classe inteira de bugs (condições de corrida, esquecimento de proteger uma variável, deadlocks) e delega ao compilador/runtime a escolha da melhor estratégia de combinação. A limitação é o escopo: só se aplica a operações de redução suportadas pela cláusula (soma, produto, mínimo, máximo, operadores lógicos/bit a bit sobre variáveis escalares), não servendo para proteger estruturas de dados complexas ou lógica condicional arbitrária.
- **atomic** é indicado quando a operação protegida é **uma única instrução simples** sobre uma variável escalar (incremento, soma, subtração, min/max, etc.). Tem overhead menor que **critical** porque usa instruções atômicas de hardware, mas ainda paga o preço da disputa pela mesma linha de cache quando muitas threads o atualizam simultaneamente com alta frequência (como ficou evidente em V4).
- **critical** (anônima) é o mecanismo mais **genérico e de maior overhead**: protege blocos de código arbitrários, com múltiplas instruções, variáveis ou até chamadas de função, mas paga o custo de um mutex de propósito geral e, pior, **todas** as regiões críticas anônimas do programa compartilham implicitamente o mesmo lock global – ou seja, uma seção crítica não relacionada a outra ainda pode bloqueá-la desnecessariamente. Deve ser reservado para blocos que realmente não podem ser expressos como **atomic/reduction**, e sempre **minimizando a frequência de entrada** (idealmente acumulando em variáveis privadas e entrando na seção crítica uma única vez por thread, como nas versões V1/V2, e nunca dentro do laço principal, como em V3).
- **critical nomeada** (`#pragma omp critical(nome)`) resolve exatamente a limitação do lock global único: quando existem **duas ou mais seções críticas protegendo dados independentes** (por exemplo, um contador de acertos e, separadamente, um log de erros ou um acumulador de estatísticas), nomear cada seção crítica permite que threads em seções com nomes diferentes executem **concorrentemente**, em vez de serem serializadas por um lock global que não tem relação com o dado que estão protegendo. Deve ser usada sempre que houver múltiplas regiões críticas logicamente independentes coexistindo no mesmo programa.

- **Locks explícitos** são a opção de **menor produtividade, porém maior controle**: úteis quando a granularidade da proteção precisa ser definida dinamicamente em tempo de execução (por exemplo, um vetor de locks, um por posição/bucket de uma tabela hash, permitindo que threads acessando posições diferentes não se bloqueiem entre si), quando é necessário tentar adquirir a proteção sem bloquear (`omp_test_lock`), ou quando o lock precisa ser mantido por mais tempo do que o permitido de forma limpa por um bloco `critical/atomic` (por exemplo, atravessando chamadas de função). O custo é a responsabilidade adicional do programador: inicializar e destruir corretamente cada lock, e evitar deadlocks ao usar múltiplos locks simultaneamente.

4.3 Roteiro proposto para escolha do mecanismo

Com base nos resultados obtidos e na discussão acima, propõe-se o seguinte roteiro de decisão, a ser percorrido nesta ordem:

1. **A operação é uma redução simples sobre uma variável escalar** (soma, produto, mínimo, máximo, and/or/xor)?
→ use `reduction`. É a opção mais rápida e mais segura, e deve ser sempre a primeira escolha quando aplicável.
2. **A operação protegida envolve múltiplas variáveis, uma estrutura de dados, ou lógica condicional não redutível a uma única instrução** (por exemplo, atualizar uma `struct`, uma lista encadeada ou um vetor de tamanho variável)?
→ use `critical`, mas **minimize sua frequência de entrada**: sempre que possível, acumule os resultados em uma variável/estrutura privada por thread durante o laço e entre na seção crítica apenas uma vez, ao final da região paralela (como em V1/V2), nunca a cada iteração (como em V3).
3. **Existem duas ou mais seções críticas protegendo dados independentes no mesmo programa?**
→ nomeie cada uma com `#pragma omp critical(nome)`, para que a disputa por uma delas não bloqueie threads que precisam apenas da outra.
4. **A operação é uma única instrução simples sobre um único escalar** (incremento, soma, min/max) e, por algum motivo, não se encaixa em `reduction` (ex.: a variável também é lida/usada de outras formas dentro da região paralela)?
→ prefira `atomic` a `critical`: mesmo semântica de exclusão mútua, porém com overhead bem menor por usar instruções atômicas de hardware.
5. **É necessário controle fino sobre a proteção?**
→ use **locks explícitos** (`omp_lock_t`), aceitando o custo adicional de gerenciá-los corretamente em troca da flexibilidade que nenhum dos mecanismos anteriores oferece.

De forma resumida: a **produtividade** decresce e a **complexidade de uso** cresce na ordem

`reduction` → `critical` (uso disciplinado) →
`atomic` → `critical` nomeada → locks explícitos

Enquanto o **desempenho**, para o caso de agregar um contador escalar como o deste experimento, tende a seguir a ordem inversa até certo ponto: **reduction** e contador privado com **critical** ao final empatam como melhores opções, seguidos por **atomic** por iteração e, por último, **critical** por iteração – confirmado empiricamente pela diferença de quase 200 vezes entre V3 e V5 para $n_pontos = 10^9$.

5 Conclusão

O experimento confirmou, de forma quantitativa, uma regra prática importante em programação paralela com OpenMP: **o mecanismo de sincronização escolhido importa menos do que a frequência com que ele é acionado**. Um **critical** usado uma vez por thread (V1/V2) é ordens de magnitude mais rápido que o mesmo **critical** usado a cada iteração de um laço com centenas de milhões de iterações (V3), mesmo protegendo exatamente a mesma informação. Trocar **critical** por **atomic** reduz o dano quando a sincronização por iteração é inevitável, mas não elimina a contenção gerada por uma variável compartilhada disputada por muitas threads.

A cláusula **reduction**, quando aplicável, é a melhor escolha tanto em desempenho quanto em produtividade, pois evita completamente a necessidade de o programador implementar manualmente a acumulação privada e a combinação final – e, neste experimento, igualou o desempenho da melhor implementação manual (V2) sem exigir nenhuma linha de código de sincronização. Mecanismos mais flexíveis, como **critical** nomeada e locks explícitos, seguem sendo ferramentas importantes para os casos em que **reduction** e **atomic** não se aplicam, mas devem ser reservados para essas situações específicas, e não usados por padrão quando alternativas mais simples e eficientes estão disponíveis.

A Código-Fonte

O código completo utilizado nesta tarefa está localizado em `../src/tarefa10.c` e é reproduzido a seguir.

```
1  /* =====
2  * Tarefa 10: Mecanismos de Sincronizacao com OpenMP
3  * Estimativa estocastica de PI (metodo de Monte Carlo)
4  *
5  * Este arquivo reimplementa o estimador da Tarefa 8
6  * (pi_monte_carlo_omp.c) para comparar 5 estrategias de sincronizacao
7  * entre threads na hora de somar os "acertos" (pontos dentro do
8  * circulo):
9  *
10 * V1) Contador PRIVADO + rand() + soma final via #pragma omp critical
11 *     (equivalente a versao 1 da Tarefa 8 - referencia/baseline)
12 *
13 * V2) Contador PRIVADO + xorshift32 + soma final via #pragma omp critical
14 *     (equivalente a versao 3 da Tarefa 8 - referencia/baseline)
15 *
16 * V3) Contador COMPARTILHADO + xorshift32 + incremento protegido por
17 *     #pragma omp critical a CADA acerto (NOVA)
18 *
19 * V4) Contador COMPARTILHADO + xorshift32 + incremento protegido por
20 *     #pragma omp atomic a CADA acerto (NOVA)
21 *
22 * V5) Acumulacao via clausula reduction(+:...) + xorshift32,
23 *     sem nenhuma diretiva explicita de exclusao mutua (NOVA)
24 *
25 * Compilar (gcc/MinGW-w64):
26 * gcc -O2 -fopenmp pi_monte_carlo_sync.c -o pi_monte_carlo_sync.exe
27 *
28 * Executar:
29 * pi_monte_carlo_sync.exe [n_inicial] [n_final] [n_threads]
30 * ===== */
31
32
33 #include <stdio.h>
34 #include <stdlib.h>
35 #include <math.h>
36 #include <time.h>
37 #include <limits.h>
38 #include <omp.h>
39
40 #define PI_REAL 3.14159265358979323846
41
42 #define MAX_THREADS 128
43
44 /* -----
45 * Gerador Xorshift32 (thread-safe)
46 * ----- */
47 static inline unsigned int xorshift32(unsigned int *state)
48 {
49     unsigned int x = *state;
50
51     x ^= x << 13;
52     x ^= x >> 17;
53     x ^= x << 5;
```

```

54     *state = x;
55     return x;
56 }
57
58
59 /* Geracao de double aleatorio */
60 static inline double random_double(unsigned int *state)
61 {
62     return (double)xorshift32(state) / (double)UINT_MAX;
63 }
64
65 /* Cria uma semente diferente por thread e NUNCA igual a zero */
66 static inline unsigned int semente_da_thread(void)
67 {
68     unsigned int base = (unsigned int)time(NULL) ^
69                         ((unsigned int)omp_get_thread_num() * 2654435761u);
70     return base | 1u;
71 }
72
73 /* =====
74 * V1: Contador PRIVADO + rand() + soma final via critical
75 * (Versão 1 da tarefa 8)
76 * ===== */
77 double pi_privado_critical_rand(long long n_pontos, int n_threads) {
78     long long total_acertos = 0;
79
80     #pragma omp parallel num_threads(n_threads)
81     {
82         long long acertos_local = 0;
83         long long i;
84
85         #pragma omp for
86         for (i = 0; i < n_pontos; i++)
87         {
88             double x = (double)rand() / (double)RAND_MAX;
89             double y = (double)rand() / (double)RAND_MAX;
90
91             if (x * x + y * y <= 1.0)
92             {
93                 acertos_local++;
94             }
95         }
96
97         /* Acumulação do total */
98         #pragma omp critical
99         {
100             total_acertos += acertos_local;
101         }
102     }
103
104     return 4.0 * (double)total_acertos / (double)n_pontos;
105 }
106
107 /* =====
108 * V2: Contador PRIVADO + xorshift32 + soma final via critical
109 * (Versão 3 da tarefa 8)
110 * ===== */
111 double pi_privado_critical_xorshift(long long n_pontos, int n_threads) {

```

```

112     long long total_acertos = 0;
113
114     #pragma omp parallel num_threads(n_threads)
115     {
116         unsigned int seed = semente_da_thread();
117         long long acertos_local = 0;
118         long long i;
119
120         #pragma omp for
121         for (i = 0; i < n_pontos; i++)
122         {
123             double x = random_double(&seed);
124             double y = random_double(&seed);
125
126             if (x * x + y * y <= 1.0)
127             {
128                 acertos_local++;
129             }
130         }
131
132         #pragma omp critical
133         {
134             total_acertos += acertos_local;
135         }
136     }
137
138     return 4.0 * (double)total_acertos / (double)n_pontos;
139 }
140
141 /* =====
142  * V3: Contador COMPARTILHADO + xorshift + critical a cada acerto
143  * ===== */
144 double pi_compartilhado_critical_xorshift32(long long n_pontos, int n_threads) {
145     long long total_acertos = 0;
146
147     #pragma omp parallel num_threads(n_threads)
148     {
149         unsigned int seed = semente_da_thread();
150         long long i;
151
152         #pragma omp for
153         for (i = 0; i < n_pontos; i++) {
154             double x = random_double(&seed);
155             double y = random_double(&seed);
156
157             if (x * x + y * y <= 1.0)
158             {
159                 #pragma omp critical
160                 {
161                     total_acertos++;
162                 }
163             }
164         }
165     }
166
167     return 4.0 * (double)total_acertos / (double)n_pontos;
168 }
169

```

```

170 /* =====
171  * V4: Contador COMPARTILHADO + xorshift + atomic a cada acerto
172  * ===== */
173 double pi_compartilhado_atomic_xorshift32(long long n_pontos, int n_threads) {
174     long long total_acertos = 0;
175
176     #pragma omp parallel num_threads(n_threads)
177     {
178         unsigned int seed = semente_da_thread();
179         long long i;
180
181         #pragma omp for
182         for (i = 0; i < n_pontos; i++) {
183             double x = random_double(&seed);
184             double y = random_double(&seed);
185
186             if (x * x + y * y <= 1.0)
187             {
188                 #pragma omp atomic
189                 total_acertos++;
190             }
191         }
192     }
193
194     return 4.0 * (double)total_acertos / (double)n_pontos;
195 }
196
197 /* =====
198  * V5: xorshift32 + clausula reduction(+:...)
199  * ===== */
200 double pi_reduction_xorshift32(long long n_pontos, int n_threads) {
201     long long total_acertos = 0;
202
203     #pragma omp parallel num_threads(n_threads)
204     {
205         unsigned int seed = semente_da_thread();
206         long long i;
207
208         #pragma omp for reduction(+:total_acertos)
209         for (i = 0; i < n_pontos; i++) {
210             double x = random_double(&seed);
211             double y = random_double(&seed);
212
213             if (x * x + y * y <= 1.0)
214             {
215                 total_acertos++;
216             }
217         }
218     }
219
220     return 4.0 * (double)total_acertos / (double)n_pontos;
221 }
222
223 /* =====
224  * Estrutura auxiliar para rodar/computar erro/imprimir cada versao
225  * ===== */
226 typedef double (*pi_func_t)(long long, int);
227

```

```

228 typedef struct {
229     const char *nome;
230     pi_func_t func;
231 } versao_t;
232
233 static void executar_versao(const char *nome, pi_func_t f, long long n_pontos, int
    n_threads) {
234     double t0 = omp_get_wtime();
235     double pi_medido = f(n_pontos, n_threads);
236     double t1 = omp_get_wtime();
237
238     double erro_abs = fabs(pi_medido - PI_REAL);
239     double erro_rel = (erro_abs / PI_REAL) * 100.0;
240
241     printf("%-12lld %-34s %-12.8f %-12.8f %-12.6f %-12.4f\n",
242           n_pontos, nome, pi_medido, erro_abs, erro_rel, t1 - t0);
243 }
244
245 /* =====
246  * MAIN
247  * ===== */
248 int main(int argc, char *argv[]) {
249     long long n_inicial = 10000;
250     long long n_final = 1000000000;
251     int n_threads = omp_get_max_threads();
252
253     if (argc >= 2) n_inicial = atoll(argv[1]);
254     if (argc >= 3) n_final = atoll(argv[2]);
255     if (argc >= 4) n_threads = atoi(argv[3]);
256
257     if (n_inicial <= 0 || n_final < n_inicial) {
258         fprintf(stderr, "Intervalo de pontos invalido.\n");
259         fprintf(stderr, "Uso: %s [n_inicial] [n_final] [n_threads]\n", argv[0]);
260         return 1;
261     }
262
263     if (n_threads <= 0) {
264         fprintf(stderr, "Numero de threads invalido.\n");
265         return 1;
266     }
267
268     if (n_threads > MAX_THREADS) {
269         fprintf(stderr, "Numero de threads limitado a %d.\n", MAX_THREADS);
270         n_threads = MAX_THREADS;
271     }
272
273     srand((unsigned int)time(NULL));
274
275     printf("=====\n");
276     printf("  TAREFA 10 - MECANISMOS DE SINCRONIZACAO COM OpenMP (PI)\n");
277     printf("=====\n");
278     printf("PI_REAL = %.15f\n", PI_REAL);
279     printf("Threads = %d\n", n_threads);
280     printf("Intervalo de n_pontos = %lld ate %lld\n\n", n_inicial, n_final);
281
282     printf("%-12s %-34s %-12s %-12s %-12s %-12s\n",
283           "N_PONTOS", "VERSAO", "PI_MEDIDO", "ERRO_ABS", "ERRO_REL(%)", "TEMPO(s)");
284     printf("-----\n");

```

```

285
286 versao_t versoes[] = {
287     { "V1 privado+rand()+critical",      pi_privado_critical_rand      },
288     { "V2 privado+xorshift+critical",    pi_privado_critical_xorshift  },
289     { "V3 compartilhado+xorshift+critical", pi_compartilhado_critical_xorshift32 },
290     { "V4 compartilhado+xorshift+atomic", pi_compartilhado_atomic_xorshift32 },
291     { "V5 xorshift+reduction",           pi_reduction_xorshift32      },
292 };
293 int n_versoes = sizeof(versoes) / sizeof(versoes[0]);
294
295 for (long long n_pontos = n_inicial; n_pontos <= n_final; )
296 {
297     for (int v = 0; v < n_versoes; v++)
298         executar_versao(versoes[v].nome, versoes[v].func, n_pontos, n_threads);
299
300     printf("-----\n");
301
302     if (n_pontos > n_final / 10) break;
303     n_pontos *= 10;
304 }
305
306 printf("\nLegenda:\n");
307 printf(" ERRO_ABS = |PI_MEDIDO - PI_REAL|\n");
308 printf(" ERRO_REL = (ERRO_ABS / PI_REAL) * 100\n");
309
310 return 0;
311 }

```

Código 1: Código-fonte completo – tarefa10.c